Ex no	Date	Experiment title	Sign
1.	20/7/26	Network commands	
2.	27/7/26	Network cables	10/8/26
3.	3/8/26	Cisco Packet Tracer	
4.	10/8/26	Setting up Local Area Network	14/8/26
5.	25/8/26	Packet capture tool : Wireshark	
6.	08/09/26	Error correction and Data link layer	
7.	18/09/26	Flow control at Data link layer	
8.	22/09/26	NMAP using Try Hacklie	
9.	24/09/26	Subnetting in Cisco Packet Tracer	
10.	25/09/26	Internetworking with routers in cisco Packet Tracer Simulator	3/10/26
11.	28/09/26	Routing at Network layer	
12.	29/09/26	End-End Communication at Transport layer	
13.	5/10/26	Implement your own ping program	
14.	6/10/26	Write a code using RAW sockets to implement packet sniffing	

Completed 13/10/26

Ex: 1
2017/26

Study of various Network commands used in Linux and windows

Aim

To study various Network commands used in Linux and windows.

Basic networking commands

1. arp -a (Fedora)

ARP is short form of address resolution protocol, It will show the IP address of your computer along with the IP address and MAC address of your router

Output

- gateway (10.0.2.2) at 52:55:0a:00:02:01 [ether] on enp0s3

2. hostname (Fedora)

This is the simplest of all TCP/IP commands. It simply displays the name of your computer

Output

fedora

3. ipconfig /all (command prompt) : This command displays detailed configuration information about your TCP/IP connection including Router, gateway, DNS, DHCP and type of ethernet adapter in your system

Output :

DESKTOP-OEABU4L (host name)

F8-ED-FC-5C-09-CF (physical address)

DHCPV6 IAID . . . . : 66645500

IPV4 Address . . . . : 172.16.10.140

Subnet Mask . . . . . : 255.255.252.0

4. Nbtstat -a. This command helps (cp) solve problems with NetBIOS name resolution. (Nbt stands for NetBIOS over TCP/IP)

Output:

Displays protocol statistics and current TCP/P connections using NBT (NetBIOS over TCP/IP)

NBTSTAT [[-a RemoteName] [-A IP address] [-C] [-n] [-r] [-R] [-RR] [-S] [-s] [interval]]

Nbtstat -n : Lists local NetBIOS names

Output

Ethernet 2:

Node IPAddress: [192.168.56.1] Scope Id: [ ]

NetBIOS Local Name Table

Name	Type	Status
DESKTOP-OEABUAL<20>	UNIQUE	Registered

...

5. netstat: (network statistics) netstat (cp) displays a variety of statistics about a computers active TCP/IP connections. It is a command line tool for monitoring network connections both incoming and outgoing as well as viewing routing tables, interface statistics etc

Output
Active connections
Proto Local Address Foreign Address State
TCP 127.0.0.1:61847 DESKTOP-OEABU4L:61848 Established

6. nslookup : (name server lookup) is a tool used to perform DNS lookups in Linux. It is used to display DNS details, such as the IP address of a particular computer, the MX records for a domain or the NS servers of a domain. nslookup can operate in two modes: interactive and non-interactive (cp)
eg: nslookup www.google.com
Output:
Server : unknown
Address : 172.16.8.1

7. pathping : combination of the Ping and Tracert. Traces the route to destination address then launches 25 second test of each router. (cp)
Output:
usage: pathping [-g host-list] [-h maximum-hops] [-i address] [-n] [-p period] [-q num-queries] [-w timeout] [-4] [-6] target-name
ping : Test connectivity between two nodes (cp)
Route : Show / manipulate IP routing table (fedora)
eg: ping www.facebook.com
Output
Ping statistics for 57.144.56.1: (www.facebook.com
Packets: Sent = 4, Received = 4, Lost = 0 (0% loss).
Pinging DESKTOP-OEABU4L [fe80::e61c:...:...]
Pinging 4.2.2.2 with 32 bytes of data:

Output for route

Manipulates network routing tables.

ROUTE [-f] [-p] [-4|-6] command [destination]

Kernal IP routing table

Destination gateway (Fedora)

default gateway .....

10.0.2.0 0.0.0.0

Some important Linux networking commands

1. ip - Setting up new systems and assigning IPs to troubleshooting existing systems. The ip command can show address information, manipulate routing, plus display network various devices, interfaces and tunnels

ip <OPTIONS> <OBJECT> <COMMAND>

-Eg. ip address show

Output :

1: lo : <LOOPBACK, UP, LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default...

t qlen 1000

2: enp0s3: ---

ip route get 10.10.1.4 - Display the route taken for IP 10.10.1.4

Output :

10.10.1.4 via 10.0.2.2 dev enp0s3 src 10.0.2.15 uid 1000 cache

2. ipconfig - The ipconfig command was a staple in many sysadmin's tool belt for configuring and troubleshooting networks. It has since been replaced by the ip command discussed above.

output:

enp0s3: flags=4163<UP, BROADCAST, RUNNING, MULTICAST> mtu 1500
inet 10.0.2.15 netmask 255.255.255.0
broadcast 10.0.2.255 ...

3. mtr - MTR (Matt's traceroute) is a program with a command-line interface that serves as a network diagnostic and troubleshooting tool. This command combines functionality of ping and traceroute commands

Syntax: mtr <options> hostname / IP

mtr google.com - The basic mtr command shows you the statistics, including each hop (hostnames) with time and loss %.

Output:

My traceroute [v0.95]
fedora (10.0.2.15) -> google.com (142.251.221.206)
Host Loss% Snt ...
1. prnaaa-ba-in-f14.1e100.net 2.7% 189

mtr -4 google.com - Show numeric IP address

Output:

mtr [options] hostname

-4 use IPv4 only
-6 use IPv6 only ....

mtr -b google.com - Show the numeric IP addresses and hostnames.

Output:

Host Loss% Snt Last ....
1. prnaaa-ba-in-f14.1e100.net 0.0% 97 6.9

mtr -c 10 google.com - Set the number of pings that you want to

Output:

My traceroute [v0.95]

fedora (10.0.2.15) -> google.com ....

Host Loss 1. Snt

4. tcpdump - Designed for capturing and displaying packets.

tcpdump -D - Interfaces tcpdump can use

Output:

1. enp0s3 [up, Running, connected]

2. any (Pseudo-device that captures on all interfaces) [up, Running]

3. lo [up, Running, Loopback] ....

dnf install -y tcpdump - command to install tcpdump

tcpdump -i enp0s3 - capture traffic on enp0s3

Output:

dropped privs to tcpdump

tcpdump: verbose output suppressed, ...

09:12:04.650915 IP6 - gateway > ff02::1:ICMP6 route advertisement, length 56 ....

tcpdup -i enp0s3 -c 10 - capture traffic to and from the host

Output:

09:17:36.131596 IP6 fedora.mdns > ff02::fb.

mbns: 0 PTR (QM)? _raop._tcp.local. (34)...

10 packets captured
10 packets received by filter
0 packets dropped by kernel.

5. ping - Ping is a tool that verifies IP-level connectivity to another TCP/IP computer by sending Internet Control Message Protocol (ICMP) Echo Request messages.

ping google.com

PING google.com (142.251.223.238) 56(84) bytes of data

64 bytes from lcmaaa-al-in-f14.1e100.net (142.251.223.238): icmp-seq=1 ttl=255 time=6.85ms

ping -c 10 google.com

PING google.com (142.250.77.110) 56(84) bytes of data.

64 bytes from pnmaaa-aq-in-f14.1e100.net (142.250.77.110): icmp-seq=1 ttl=255 time=4.25ms

Configuring an Ethernet connection by using nmcli

If you connect a host to the network over Ethernet, you can manage the connection's settings on the command line by using the nmcli utility.

Procedure

1. List the NetworkManager connection profiles

nmcli connection show

NAME	UUID	TYPE	DEVICE
Wired connection 1	06726542-1f12-3ba6...	ethernet	enp0s

2. nmcli connection add con-name <connection-name> ifname <device-name>
Modify an existing profile
3: Reriane the connection profile
nmcli connection modify "wired connection 1"
4. Display current settings of connection profile. nmcli connection show

NAME	UUID	TYPE	DEVICE
enp0s3	a10fd672-888a-33f4-bb5f-576089bceaf6	--	
lo	8ad4	--	

5. Configure the IPV4 settings:
To use DHCP, enter
nmcli connection modify "Wired connection 1"
ipv4.method auto
Skip if ipv4 is set auto
To set a static IPV4 address, network mask, default gateway, DNS servers and search domain, enter
nmcli connection modify "Wired connection 1"
ipv4.method manual ipv4.addresses 192.0.2.1/24 ipv4.gateway 192.0.2.254 ipv4.dns 192.0.2.200 ipv4.dns-search example.com.
6. Configure the IPV6 settings
To use stateless address autoconfiguration (SLAAC) enter
nmcli connection modify "Wired connection 1"
ipv6.method.auto

Activate the profile

nmcli connection up Internal-LAN

connection successfully activated (D-Bus active path: /org/freedesktop/NetworkManager/ActiveConnection/3)

3/8/24

Result

Thus various commands used in linux and windows is executed successfully

3/8/24

ex: 2
27/7/26

Study of Network cables

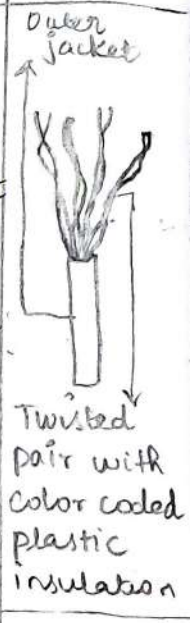
Aim

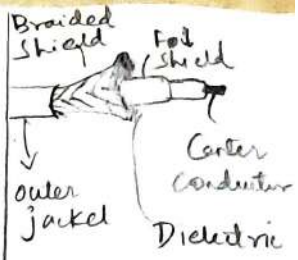
Study of different types of network cables.

a) understand different types of network cable

Different type of cables used in networking are

1. Unshielded Twisted Pair (UTP) cable.
2. Shielded Twisted Pair (STP) cable.
3. coaxial cable
4. Fibre optic cable

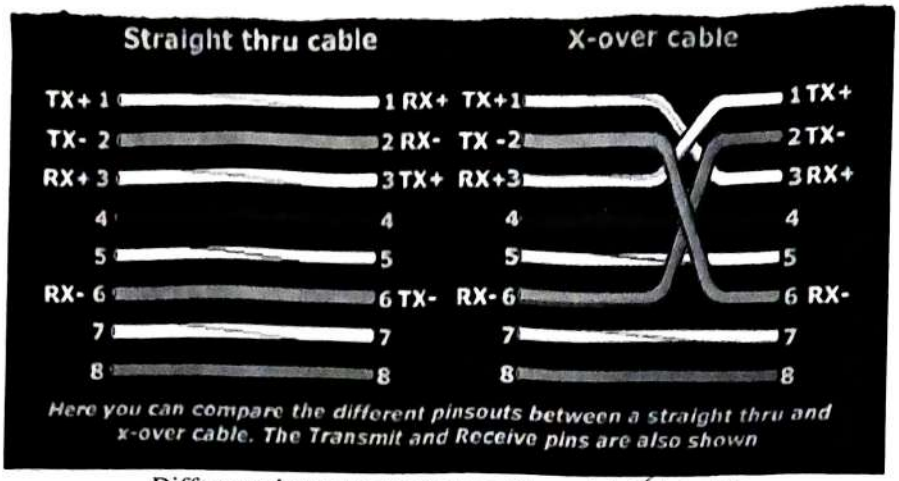
cable type	category	Maximum Data Transmissi	Advantages/ Disadvantages	Application/ use	Image
UTP	category 3	10bps	Advantages: cheaper in cost Easy to install as they have a smaller overall diameter Disadvantages More prone to (EMI) Electromagnetic interference and noise	10 Base-T Ethernet Fast Ethernet Gigabit Ethernet Fast Ethernet, Gigabit Ethernet	
	category 5	upto 100 Mbps			
	category 5e	1 Gbps			
STP	category 6, 6a	10Gbps	Advantages: Shielded Faster than UTP. Less. Susceptible to noise and interference Disadvantages Expensive Greater installation	Gigabit Ethernet 10G Ethernet (55m) widely used in data centres Gigabit Ethernet, 10G Ethernet (100 m)	
SSTP	category 7	10Gbps			

Coaxial cable	RG-6 RG-59 RG-11	10-100 Mbps	Advantages: High bandwidth Immune to interference Low loss bandwidth Versatile Disadvantages Limited distance cost size is bulky	Speed of signal is 500m Television network High speed internet connections	
Fibre optics cable	Single mode Multi mode	100Gbps	Advantages High speed High bandwidth High security Long distance Disadvantages Expensive Requires skilled installers	Maximum distance of fibre optics cable is around 100 meters	

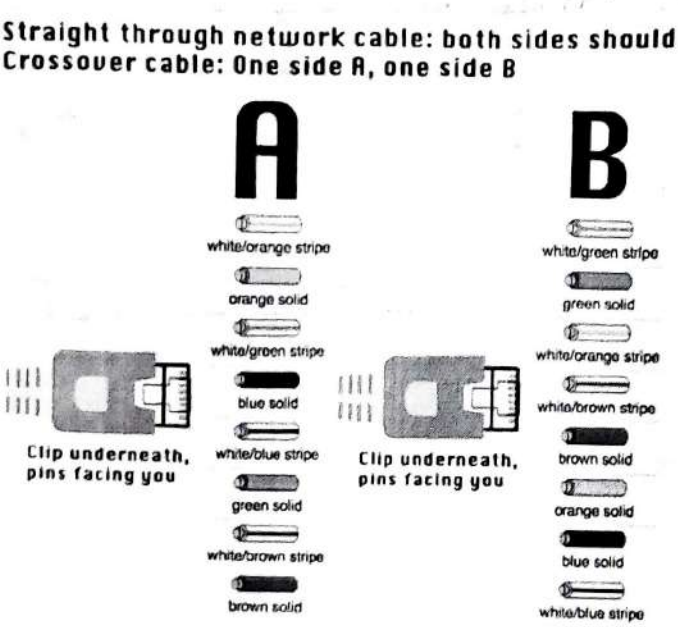
b) Make your own Ethernet Cross-Over Cable / Straight cable

Tools and parts needed:

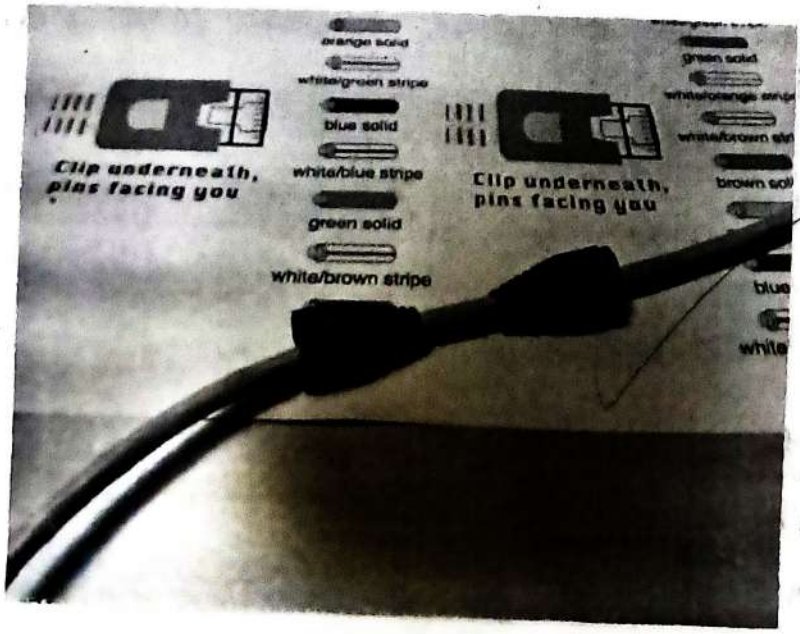
* Ethernet cabling, CAT5e is certified for gigabit support, but CAT 5 cabling works as well, just over short distances
* A crimping tool. This is an all-in-one networking tool shaped to push down the pins in the plug and strip and cut the shielding off the cables
* Two RJ45 plugs.
* Optional two plug shields



Difference between crossover cable and straight cable



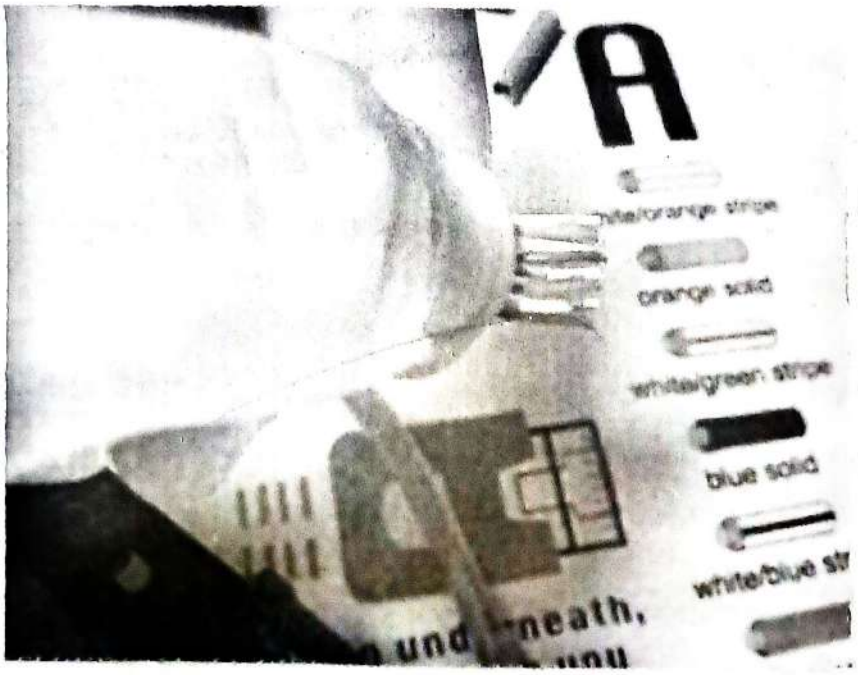
Step 1: To start construction of the device, begin by threading shields onto the cable



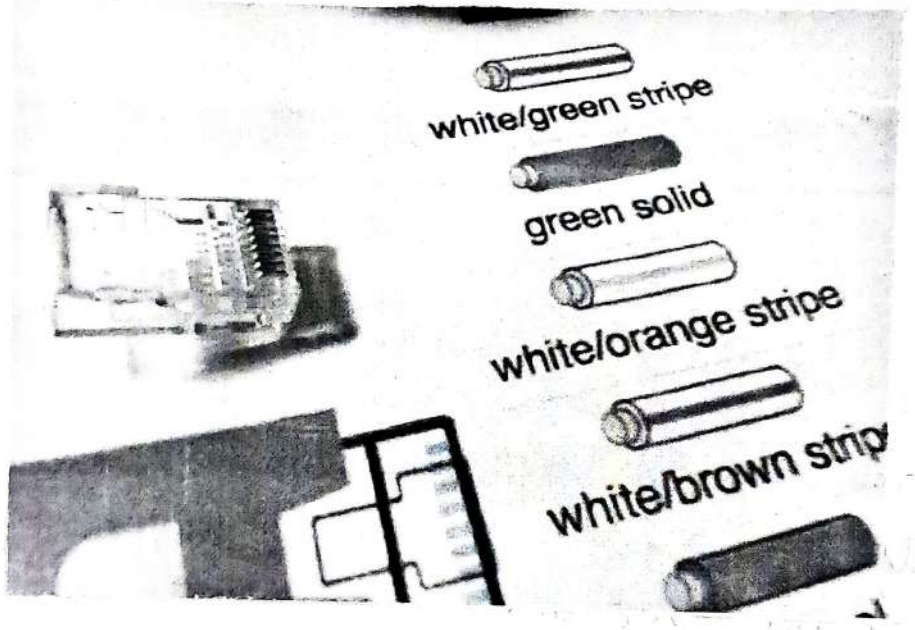
Step 2. Next, Strip approximately 1.5cm of cable shielding from both ends. The crimping tool has a round area to complete this task



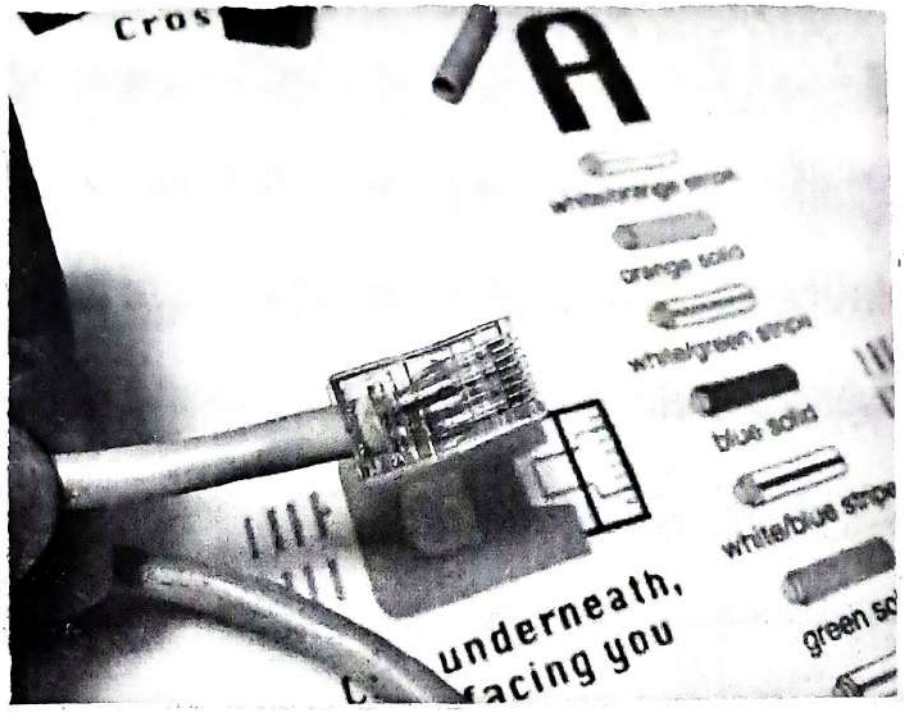
Step 3: After, you will need to untangle the wires; there should be four "twisted pairs". Referencing back to the sheet, arrange them from top to bottom. One end should be in arrangement A and the other in B.



Step 4: Once the order is correct, bunch them together in a line, and if there are any that stick out farther than others, snip them back to create an even level. The difficult aspect is placing these into the RJ45 plug without messing up the order. To do so, hold the plug with the clip side facing away from you and have the gold pins facing toward you.



Step 5: Next, push the cable right in. The notch at the end of the plug needs to be just over the cable shielding, and if it isn't, that means that you stripped off too much shielding. Simply snip the cable back a little more.



Step 6 : After the wires are securely sitting inside the plug, insert it into the crimping tool and push down. It should be shaped correctly, but pushing too hard can crack the fragile plastic plug.

Step 7: Lastly, repeat for the other end using crossover cables to straight through cable. To test it, plug it in and attempt to connect two devices directly.

Result

Thus the different types of network cables were studied

31/8/26

3/8/26 Ex:3 Experiments on CISCO PACKET TRACER

Aim:

To study the Packet tracer tool installation and user Interface overview

Introduction:

A simulator, as the name suggests, simulates network devices and its environment. Packet Tracer is an exciting network design, simulation and modelling tool.

1. It allows you to model complex systems without the need for dedicated equipment
2. It helps you to practice your network configuration, and troubleshooting skills via computer or an Android or iOS based mobile device.
3. It is available for both the Linux and Windows desktop environments
4. Protocols in Packet Tracer are coded to work and behave in the same way as they would on real hardware

User Interface Overview:

Menu bar - This is a common menu found in all software applications; it is used to open, save, print, change preferences, and so on

Main toolbar - This bar provides shortcut icons to menu options that are commonly accessed, such as open, save, zoom, undo and redo, and on the right-hand side is an icon for entering network information for the current network.

Logical/Physical workspace tabs - These tabs allow you to toggle between the Logical and Physical work area.

Workspace - This is the area where topologies are created and simulations are displayed

Common tools bar - This toolbar provides controls for manipulating topologies, such as select, move layout, place note, delete, inspect, resize shape, and add simple/complex PDU.

Real-time/Simulation tabs - These tabs are used to toggle between the real and simulation modes. Buttons are also provided to control the time, and to capture the packets.

Network component box - This component contains all of the network and end devices available with Packet tracer, further divided into Area 7a, Area 7b

User-created packet box - Users can create highly-customized packets to test topology

Analyse the behaviour of network devices using CISCO PACKET TRACER simulator

1. From the network component box, click and drag-and-drop the below components.

a. 4 Generic PCs and One HUB

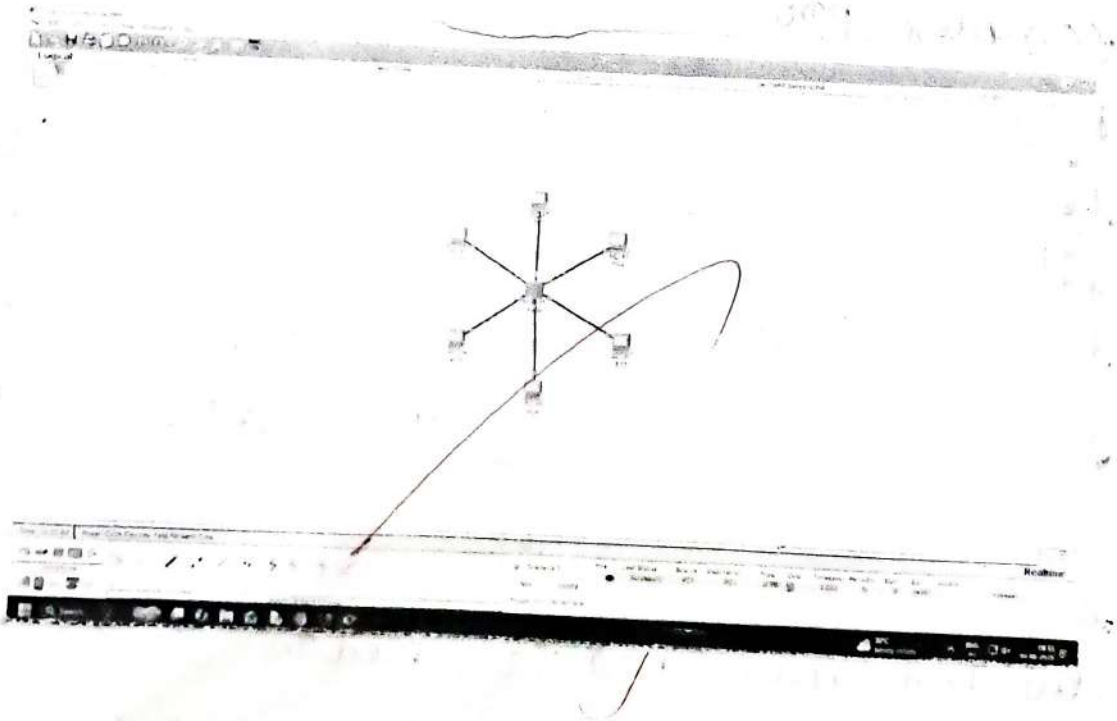
b. 4 Generic PCs and One switch

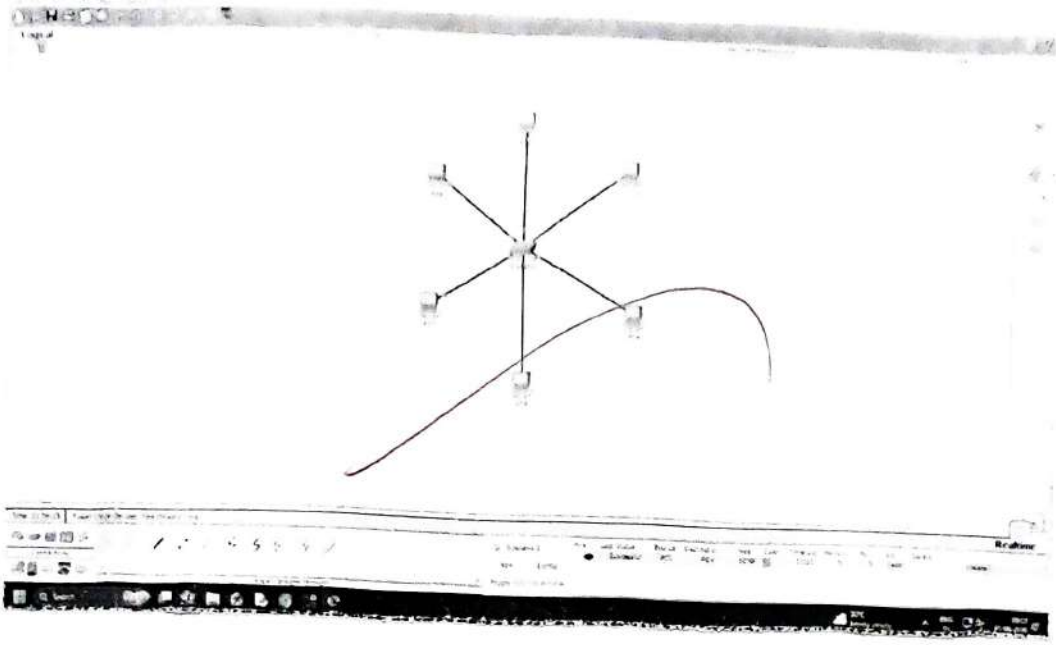
2. Click on connections.

a. Click on copper-Straight-Through cable

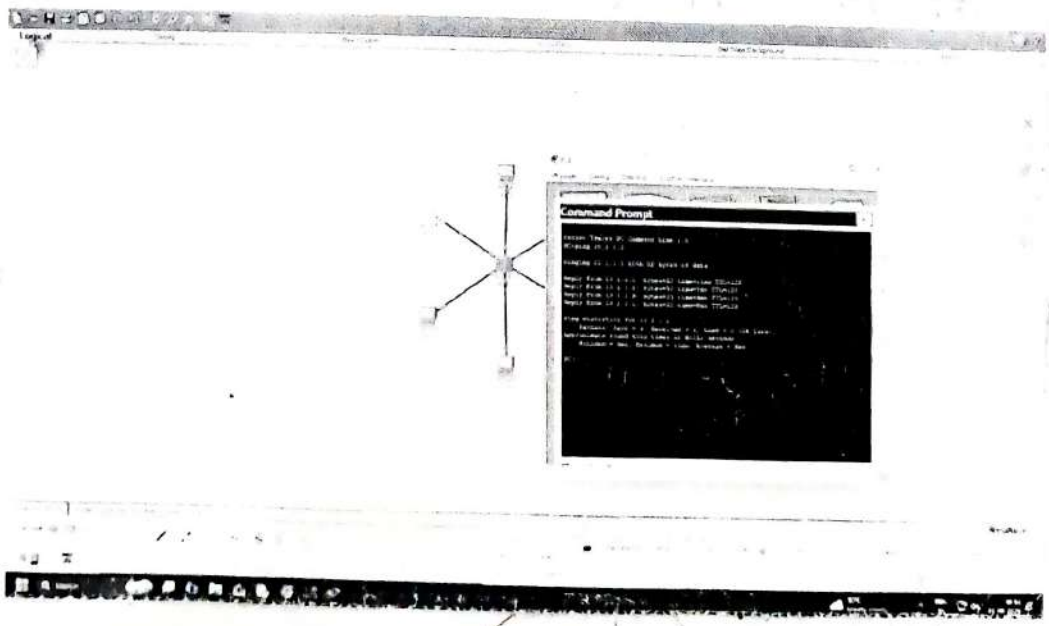
b. Select one of the PC and connect it to HUB using the cable. The link LED should glow in green, indicating that the link is up. Similarly connect remaining 3 PCs to HUB

c. Similarly connect 4 PCs to the switch using copper straight-through cable.





3. Click on the PCs connected to hub, go to the Desktop tab, click on IP configuration and enter an IP address and subnet mask. Here, the default gateway and DNS server information is not needed as there are only two end devices in the network





Click on the PDU (message icons) from common tool bar

3. Drag and drop it on one of PC (source machine) and then drop it on another PC (destination machine) connected to the HUB.

4. Observe the flow of PDU from source PC to destination PC by selecting the Realtime mode of simulation.

5. Repeat step #3 to step #5 for the PCs connected to the switch.

6. Observe how HUB and switch are forwarding the PDU and write your observation and conclusion about the behaviors of Switch and HUB.

Result

Thus we experimented CISCO PACKET TRACER.

10/8/26 Ex 4 Setting up a Local Area Network (LAN)

Aim:

To design, set up and configure a local Area Network (LAN) connecting three computers and server through a network switch, assign static IP addresses to all devices, verify connectivity using the ping command, and demonstrate resources/data sharing across the network.

Equipment Required

Equipment	Quantity	Specification
Personal Computers (PC)	3	Ethernet (NIC) enabled
Network Switch	1	8/16/24 port, unmanaged/managed
Ethernet cables (cat 5e/cat 6)	4	Straight-through, RJ45 connectors
Server	1	Ethernet (NIC) enabled

Theory

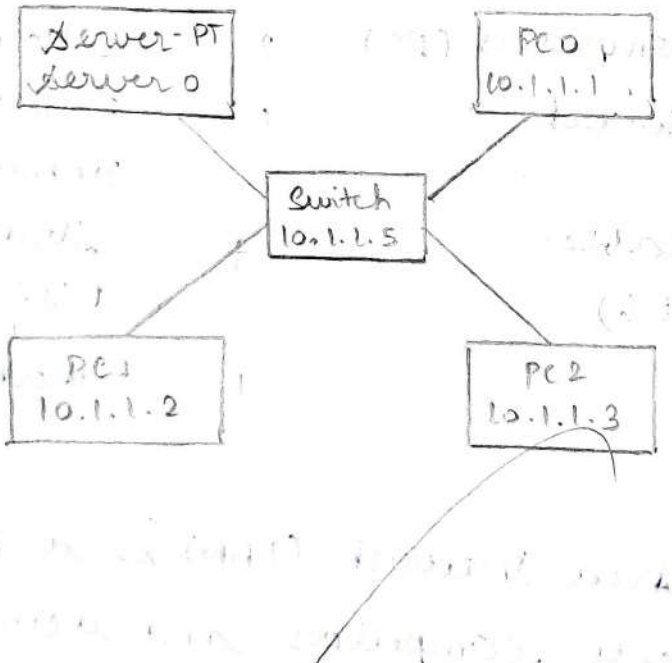
A Local Area Network (LAN) is a network that connects computers and devices within a limited geographical area such as home, office, or laboratory, allowing them to share resources such as files, printers, and internet connections.

A star topology is used in this experiment where all end devices are individually

connected to a central network switch. The switch operates at Layer 2 of the OSI model and forwards data frames between connected devices based on their MAC addresses. Every device on a LAN must be assigned an IP address within the same network range to be able to communicate with one another.

Network diagram

The LAN was implemented using a star Topology with 3PCs, server connected individually to a central switch.



IP Address Configuration Table

Device	IP Address	Subnet Mask	Connected To
PC 0	10.1.1.1	255.0.0.0	Switch Port Fa0/1
PC 1	10.1.1.2	255.0.0.0	Switch Port Fa0/2
PC 2	10.1.1.3	255.0.0.0	Switch Port Fa0/3
Server	10.1.1.4	255.0.0.0	Switch Port Fa0/4
Switch (Management)	10.1.1.5	255.0.0.0	VLAN1 Interface

Implementation in Cisco Packet Tracer

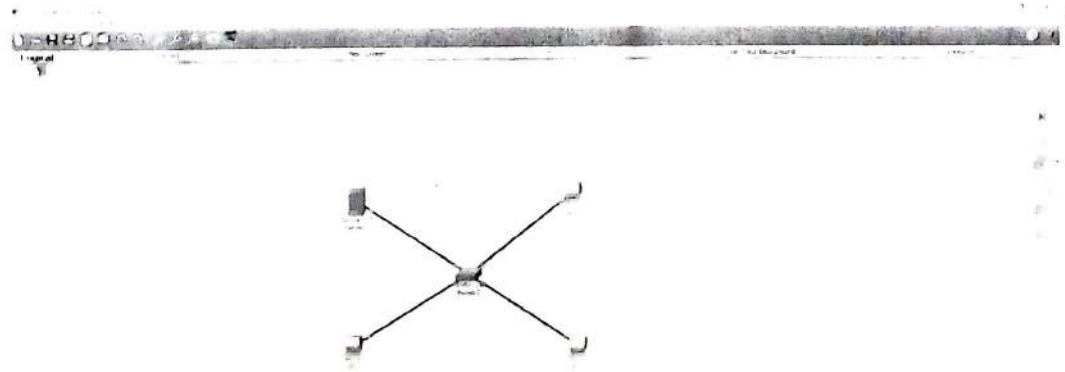
Since physical hardware was limited, the LAN was also implemented and verified using Cisco Packet Tracer simulation software, following the steps below

Step 1: Place the Devices

1. Open Cisco Packet Tracer
2. From the device category panel, select End devices and drag 3 PCs and server onto the workspace.
3. Select switches and drag 1 switch onto the workspace.

Step 2: Cable the Devices

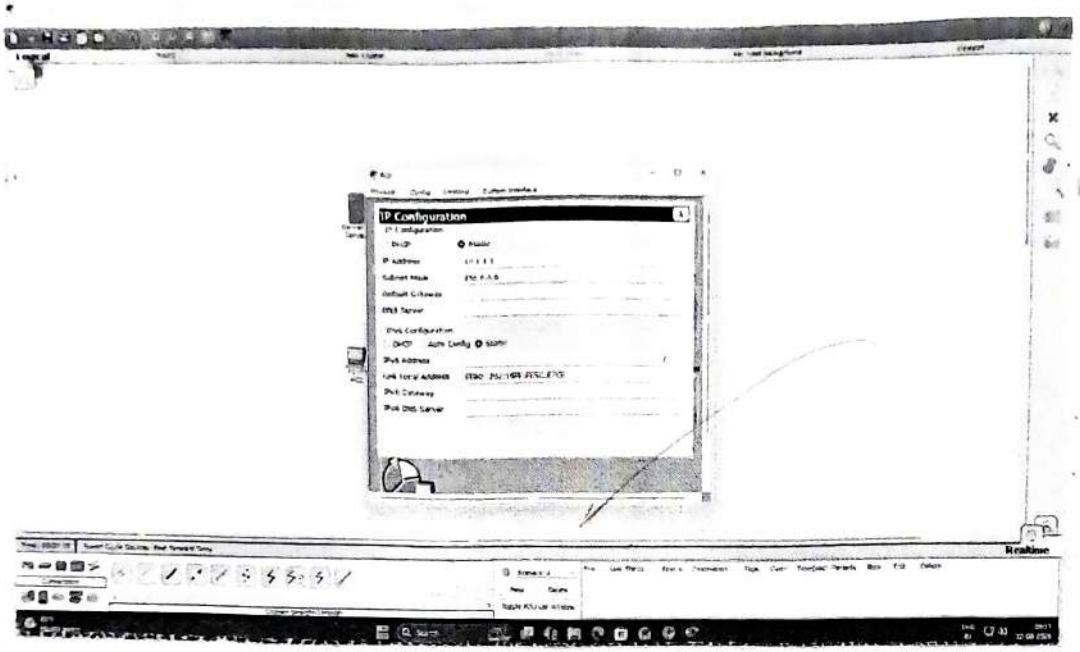
4. Click Connections → select Copper Straight-Through cable.
5. Click each PC, then click the switch, and select an available FastEthernet port to complete each connection
6. Repeat for all PCs and server. Green link lights confirm a successful physical connection





Step 3: Assign IP address to PCs.

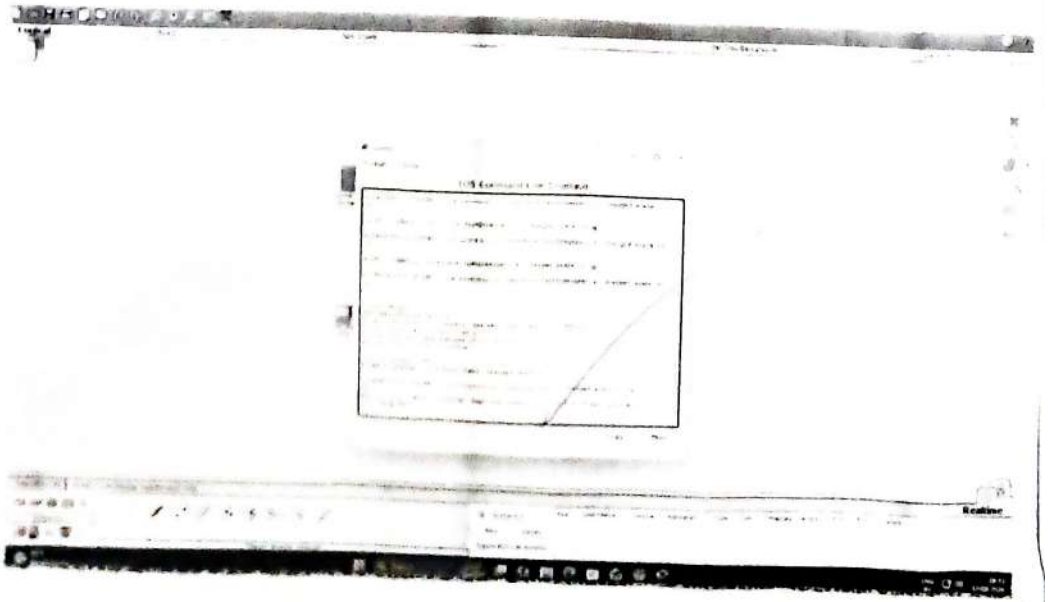
7. Click a PC → Desktop tab → IP configuration
8. Enter the static IP Address and Subnet Mask as per the IP table for each PC.



Step 4: Configure the Switch Mangement IP.

9. Click the switch → CLI tab
10. Enter the following commands to assign a management IP.

```
enable
configure terminal
interface vlan 1
ip address 10.1.1.5 255.0.0.0
no shutdown
exit
```

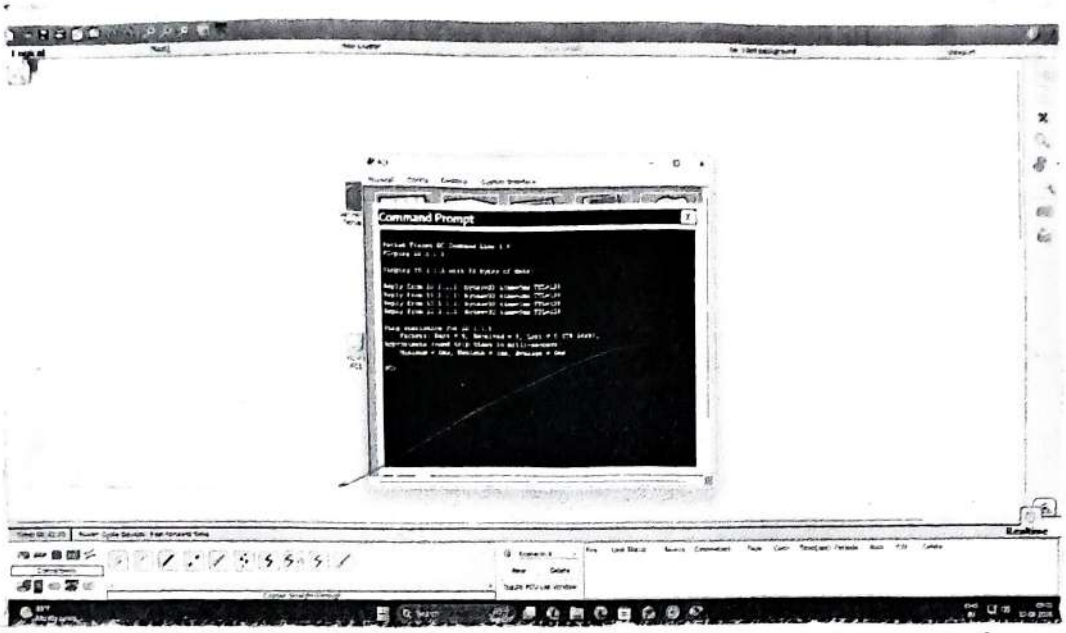


Step 5: Verify connectivity using Ping

11. Click a PC → Desktop tab → Command Prompt

12. Use the ping command to test connectivity to the other PCs. eg. ping 10.1.1.2

13. Successful "Reply from..." messages confirm connectivity



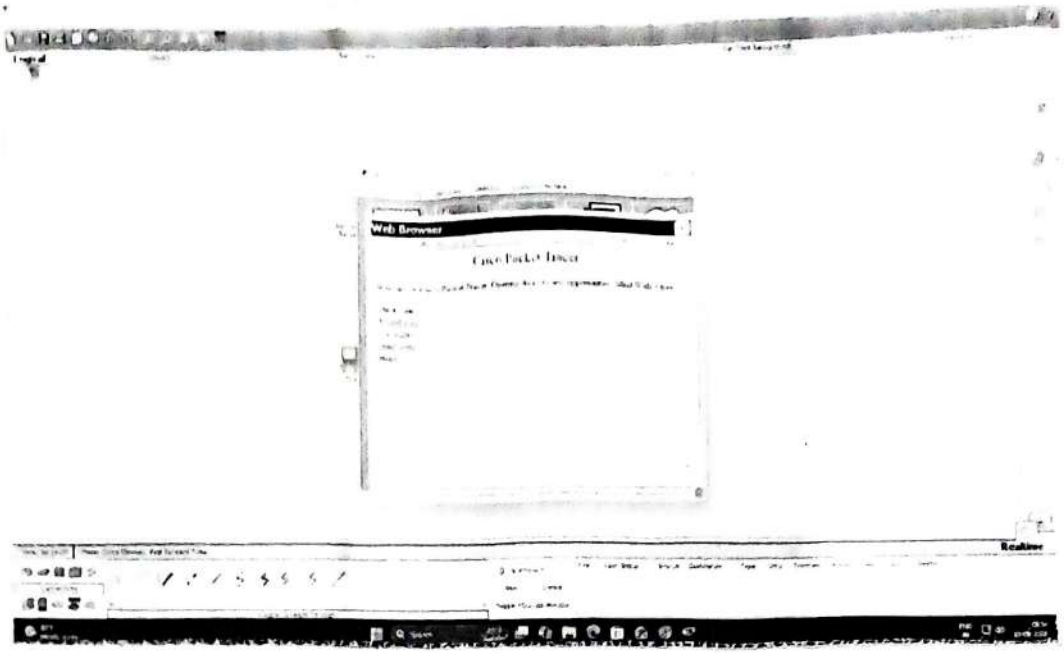
Step 6: Demonstrate Data sharing (Server simulation)

14. Add a server device (End Devices → Server-PT) and connect it to the switch

15. Assign it an IP address (eg. 10.1.1.10, subnet mask 255.0.0.0) via its Desktop → IP configuration tab

16. Open the server's Services (config) tab and enable the HTTP service

17. From another PC, open the Web Browser (Desktop tab) and enter the server's IP address to load the page, confirming successful data transfer across the LAN



Step 7: Observe Packet Flow in Simulation Mode

18. Switch to Simulation mode

19. Send ping, use play/capture, observe MAC-address-based forwarding

Result

All PCs, server and server and switch are connected, ping returned successful. The shared folder was successfully accessed from other computer on the network.

25/8/26 Ex: 5 Experiments on Packet capture tool: Wireshark

Aim:

To understand the features of wireshark as a packet capture tool and understand encapsulation of information at various layers of a Protocol stack

Packet Sniffer

Sniffs messages being sent/received from/by your computer

Store and display the contents of the various protocol fields in the messages.

Passive program

- never sends packets itself.
- no packets addressed to it.
- receives a copy of all packets (sent/received)

Packet Sniffer structure Diagnostic Tools

* Tcpdump

eg. tcpdump -enx host 10.129.41.2 -w exe3.out

* Wireshark

- wireshark -r exe3.out

Capturing and analysing packets using Wireshark Tool

To filter, capture, view, packets in Wireshark tool. Capture 100 packets from the Ethernet: IEEE 802.3 LAN Interface and save it.

Procedure

- Select Local Area connection in Wireshark.
- Go to capture option
- Select stop capture automatically after 100 packets.
- Then click start capture
- Save the packets



1. Create a filter to display only TCP / UDP packets and provide the flow graph

Procedure

Select Local Area Connection in Wireshark

Go to capture option.

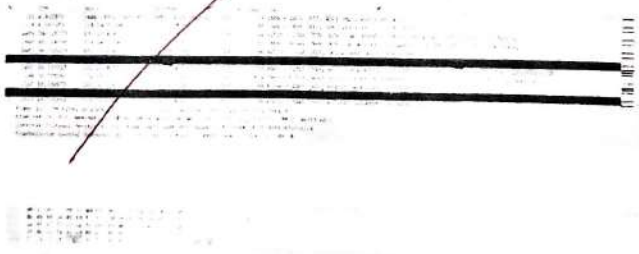
Select stop capture automatically after 100 packets.

Then click start capture.

Search TCP packets in search bar.

To see flow graph click Statistics Flow graph.

Save the packets.





2. Create a filter to display only ARP packets and inspect the packets.

Procedure

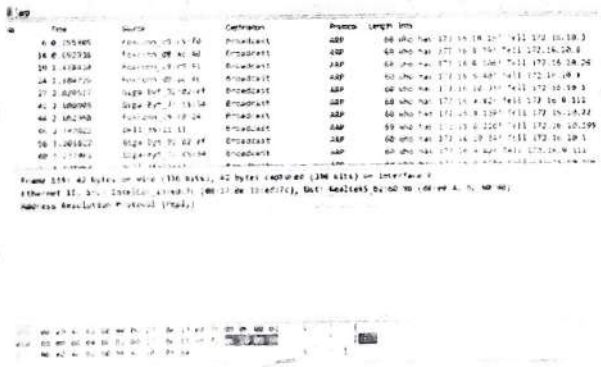
Go to capture option

Select stop capture automatically after 100 packets.

Then click start capture.

Search ARP packets in search bar.

Save the packets.



3. Create a filter to display only DNS packets and provide the flow graph.

Procedure

Go to capture option

Select stop capture automatically after 100 packets.

Then click start capture

Search DNS packets in search bar.

To see flow graph click Statistics Flow graph.



Save the packets.



4. Create a Filter to display only HTTP packets and inspect the packets

Procedure

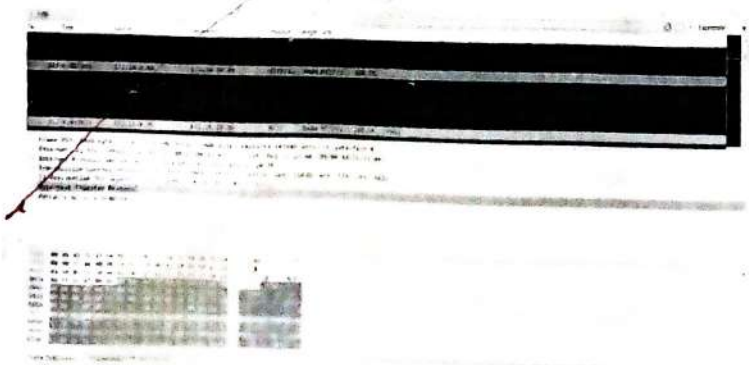
Select Local Area Connection in wireshark.

Go to capture option

Select stop capture automatically after 100 packets.

Then click start capture

Search HTTP packets in search bar



save the packets.

5. Create a Filter to display only IP/ICMP packets and inspect the packets

Procedure

Select Local Area connection in Wireshark.

Go to Capture option.

Select stop capture automatically after 100 packets.

Then click Start capture

Search ICMP/IP packets in search bar.



6. Create a Filter to display only DHCP packets and inspect the packets.

Procedure.

Select Local Area Connection in Wireshark.

Go to capture option

Then click capture.

Search DHCP packets in search bar and save



Result

Thus Experiment on Packet capture tool Wireshark was done successfully.

28/09/26

Exercise 6:

Error correction at Data Link Layer

Aim:

Write a program to implement error detection and correction using HAMMING code concept. Make a test run to input data stream and verify error correction feature.

Error correction at Data Link Layer

Hamming code is a set of error-correction codes that can be used to detect and correct the errors that can occur when the data is transmitted from the sender to the receiver. It is a technique developed by R.W. Hamming for error correction.

Create sender program with below features

1. Input to sender file should be a text of any length. Program should convert the text to binary.
2. Apply hamming code concept on the binary data and add redundant bits on to it.
3. Save this output in a file called channel.

Create a receiver program with below features

1. Receiver program should read the input from channel file.
2. Apply hamming code on the binary data to check for errors.
3. If there is an error, display the position of the error.
4. Else remove the redundant bits and convert the binary data to ascii and display output

sender.py

```
def calculate_parity_bits(data_length):
    r=0
    while (2**r)<(data_length+r+1):
        r+=1
    return r
def hamming_encode(data):
    m=len(data)
    r=calculate_parity_bits(m)
    n=m+r
    hamming=['0']*(n+1)
    j=0
    for i in range(1,n+1):
        if(i&(i-1))!=0:
            hamming[i]=data[j]
            j+=1
    for p in range(r):
        position=2**p
        parity=0
        for i in range(1,n+1):
            if i & position:
                parity^=int(hamming[i])
        hamming[position]=str(parity)
    return ''.join(hamming[1:])
with open("input.txt","r") as file:
    text=file.read
print("Input text:")
print(text)
binary_data=""
for character in text:
    binary_data+=format(ord(character),"08b")
print("\n Binary data:")
```

```
print(binary_data)
encoded_data = ""
for i in range(0, len(binary_data), 8):
    block = binary_data[i:i+8]
    if len(block) == 8:
        encoded_data += hamming_encode(block)
print("\nHamming encoded data:")
print(encoded_data)
with open("channel.txt", "w") as file:
    file.write(encoded_data)
print("\nHamming data stored in channel.t
```

receiver.py

```
def hamming_decode(hamming):
    n = len(hamming)
    r = 0
    while (2**r) <= n:
        r += 1
    error_position = 0
    for p in range(r):
        position = 2**p
        parity = 0
        for i in range(1, n+1):
            if i & position:
                parity ^= int(hamming[i-1])
        if parity:
            error_position += position
    if error_position == 0
        print("No error detected.")
    else:
        print("Error detected!")
```

```
        print("Error position:", error_position)
        if error_position <= n:
            hamming = list(hamming)
            index = error_position - 1
            hamming[index] = '1' if (hamming[index]
            == '0' else '0'
            hamming = ''.join(hamming)
            print("Error corrected.")
        else:
            print("Invalid error position.")
    data = ""
    for i in range(1, n+1):
        if (i & (i-1)) != 0:
            data += hamming[i-1]
    return data
with open("channel.txt", "r") as file:
    received_data = file.read().strip()
print("Received Hamming data:")
print(received_data)
binary_data = ""
for i in range(0, len(received_data), 12):
    block = received_data[i:i+12]
    if len(block) == 12:
        binary_data += hamming_decode(block)
print("\nRecovered binary data:")
print(binary_data)
output = ""
for i in range(0, len(binary_data), 8):
    byte = binary_data[i:i+8]
```

```
        if len(byte) == 8:
            output += chr(int(byte, 2))
print("\nOutput text:")
print(output)
```

input.txt

HELLO

Output :



channel.txt :

000110011000110010000101010110001100010110001100100010001111



Result :

Thus a program to implement error detection and correction using Hamming code concept was written and executed successfully.

18/09/24

Exercise 7:

Flow control at Data Link Layer

Aim:

Write a program to implement flow control at data link layer using SLIDING WINDOW PROTOCOL. Simulate the flow of frames from one to another

sender.py

```
import time
import os
message = input("Enter the text message: ")
window_size = int(input("Enter window size: "))
frames = [[i+1, ch] for i, ch in enumerate
(message)]
base = 0
while base < len(frames):
    end = min(base + window_size, len(frames))
    with open("Sender_Buffer.txt", "w") as f:
        for i in range(base, end):
            f.write(f"Frame No: {frames[i][0]},
            DATA: {frames[i][1]}\n")
    print("\nFrames sent:")
    for i in range(base, end):
        print(f"Frame No: {frames[i][0]},
        DATA: {frames[i][1]}")
    print("\nWaiting for acknowledgement...")
    time.sleep(3)
    while not os.path.exists("Receiver_Buffer.txt")
        time.sleep(1)
```

```
with open("Receiver_Buffer.txt","r") as f:
    response = f.read().strip()
print("Receiver message:", response)
if response.startswith("ACK"):
    ack_no = int(response.split(":")[1].
    strip())
    if ack_no == end:
        base = end
        os.remove("Receiver Buffer.txt")
    else:
        print("Unexpected ACK Resending from
        os.remove("Receiver_Buffer.txt")
elif response.startswith("NACK"):
    nack_no= int(response.split(":")[1].strip(
    print(f"NACK received for Frame {nack_n
    Resending from Frame {nack_no}...")
    base= nack_no-1
    os.remove("Receiver_Buffer.txt")
print("\n All frames transmitted successfully.")
```

receiver.py

```
import time
import os
expected_frame = 1
while True:
    if os.path.exists("Sender_Buffer.txt"):
        with open("Sender_Buffer.txt","r") as f:
            lines = f.readlines()
        if not lines:
            time.sleep(1)
            continue
```

```
print ("\n Received frames: ")
error = False
for line in lines:
    parts = line.strip().split(",")
    frame_no = int(parts[0].split(":")[1].
    strip())
    data = parts[1].split(":")[1].strip()
    print(f"Frame No: {frame_no}, DATA:
    {data}")
    if frame_no != expected_frame:
        print(f" error detected. Expected
        Frame {expected_frame}, received
        Frame {frame_no}")
        with open("Receiver_Buffer.txt", "w") as
        f:
            f.write(f"NACK: {expected_frame}")
        error = True
        break
    expected_frame += 1
if not error:
    with open("Receiver_Buffer.txt", "w") as
        f.write(f"ACK: {expected_frame-1}")
os.remove("Sender_Buffer.txt")
time.sleep(1)
```

```
PS C:\Users\bhuvi\Downloads\Practical-7> python receiver.py

Received frames:
Frame No: 1, DATA: H
Frame No: 2, DATA: E
Frame No: 3, DATA: L

Received frames:
Frame No: 4, DATA: L
Frame No: 5, DATA: O
```

```
PS C:\Users\bhuvi\Downloads\Practical-7> python sender.py
Enter the text message: HELLO
Enter window size: 3

Frames sent:
Frame No: 1, DATA: H
Frame No: 2, DATA: E
Frame No: 3, DATA: L

Waiting for acknowledgement...
Receiver response: ACK: 3

Frames sent:
Frame No: 4, DATA: L
Frame No: 5, DATA: O

Waiting for acknowledgement...
Receiver response: ACK: 5

All frames transmitted successfully.
```

Result

Thus a program to implement flow control at data link layer using sliding window was executed successfully.

22/09/26

Exercise 8:

Nmap Live Host Discovery

Aim:

To discover live hosts on a network using Nmap ARC, ICMP, TCP and UDP host-discovery scans and understand the protocols used at different TCP/IP layers.

Introduction

Nmap (Network Mapper) is a free and open-source network scanning tool used for host discovery, network mapping and service detection.

Before performing a port scan, it is useful to determine which systems are active. Nmap provides different host-discovery techniques depending on the network and protocol being used.

Main Host Discovery Methods

Method	Option	Protocol	Layer
ARP	-PR	ARP	Link
ICMP echo	-PE	ICMP	Network
ICMP Timestamp	-PP	ICMP	Network
ICMP Address Mask	-PM	ICMP	Network
TCP SYN	-PS	TCP	Transport
TCP ACK	-PA	TCP	Transport
UDP	-PU	UDP	Transport

Requirements

- Try Hackme Nmap Live Host Discovery room
- Try Hackme AttackBox or Kali Linux
- Nmap
- Basic knowledge of TCP/IP and subnetting

1. Target specification

Nmap can scan individual IP addresses, ranges, or complete subnets

nmap 10.10-1.5

nmap 10.10.1.15-20

nmap 10.10.1.0/24

To list targets without actually scanning them

nmap -SL 10.10.1.0/24

To disable DNS resolution

nmap -n 10.10.1.0/24

2. ARP Host Discovery

ARP (Address Resolution Protocol) works at the Link Layer and is used to find hosts on the same local subnet.

The attacker sends an ARP broadcast asking

"Who has this IP address?"

An active host replies with its MAC address

command:

sudo map -PR -sn 10.200.6.0/24

-PR → ARP discovery

-sn → Host discovery only; no port scan

Result

Nmap scan report for 10.200.6.1

Host is up

Nmap done: 256 IP addresses (1 host up)

3. ICMP Host Discovery

ICMP works at the Network Layer. Nmap can use different ICMP request types to discover active hosts.

a) ICMP Echo

sudo nmap -PE -sn 10.200.6.0/24

b) ICMP Timestamp

nmap -PP -sn 10.200.6.0/24

c) ICMP Address Mask

nmap -PM -sn 10.200.6.0/24

Returns:

Nmap done: 256 IP addresses (0 hosts up)

4. TCP Host Discovery

TCP ping can be used when ICMP is blocked

a) TCP SYN Ping

nmap -PS -sn 10.200.6.0/24

b) TCP ACK Ping

sudo nmap -PA -sn 10.200.6.0/24

5. UDP Host Discovery

UDP ping sends a UDP packet to a target port.

sudo nmap -PU -sn 10.200.6.0/24

If the port is closed, the host normally responds with an ICMP Port unreachable message.

6. Default Nmap Host Discovery

When no specific discovery option is provided, Nmap automatically selects suitable techniques.

For example

Privileged + local network : ARP is normally used

Privileged + remote network : ICMP Echo, TCP ACK

Unprivileged + remote network : TCP - based discovery is used.

Host discovery only: nmap -sn 10.200.6.0/24

Skip host discovery: nmap -Pn 10.200.6.0/24

7. Reverse DNS

Reverse DNS converts an IP address into a hostname.

Disable DNS lookup: nmap -n 10.200.6.0/24

Force DNS lookup: nmap -R 10.200.6.0/24

Result

The Nmap host-discovery techniques were successfully studied. ARP is useful for discovering hosts on local subnet, while ICMP, TCP, and UDP techniques can be used when ARP is not applicable.

24/09/26

Exercise - 9

Implementation of SUBNETTING in CISCO PACKET TRACER simulator

Aim:

Implementation of SUBNETTING in CISCO PACKET TRACER simulator

classless IP subnetting is a technique that allows for more efficient use of IP addresses by allowing for subnet masks that are not just the default masks for each IP class. This means that we can divide our IP address space into smaller subnets, which can be useful when we have a limited number of IP addresses but need to create multiple networks.

CREATING A NETWORK TOPOLOGY

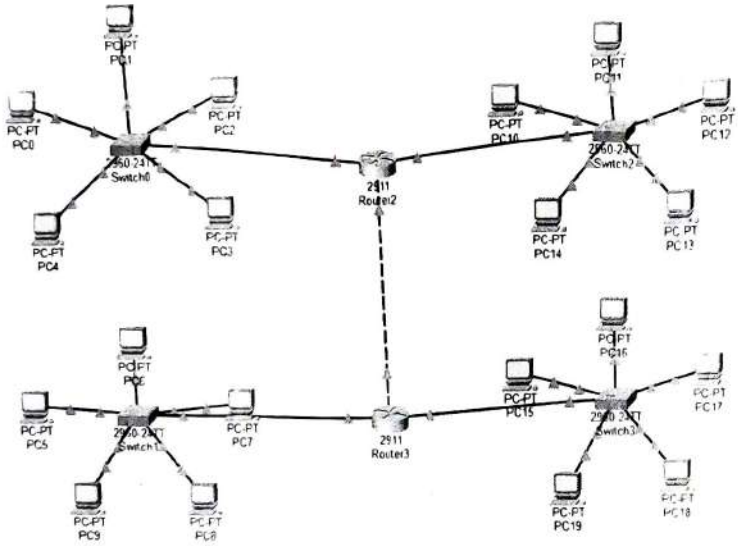
The first step in implementing classless IP subnetting is to create a network topology in Packet Tracer. To create a network topology in Packet Tracer, select the "New" button in the top left corner, select "Network" and "Generic". This will create a blank network topology that we can use to add devices

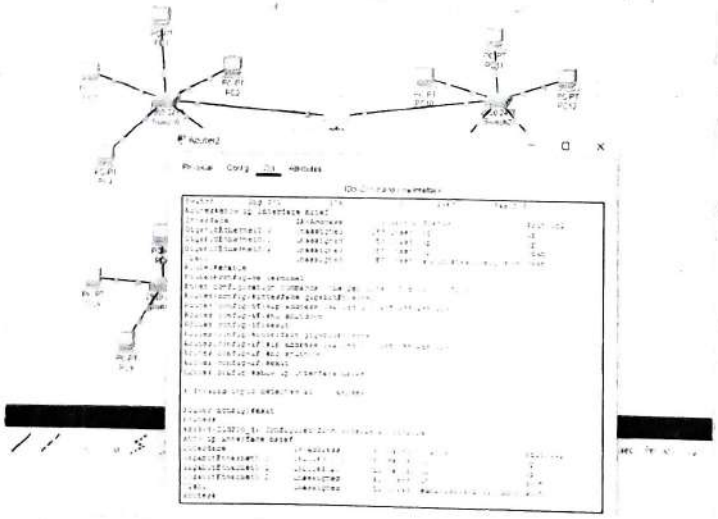
ADDING THE DEVICES

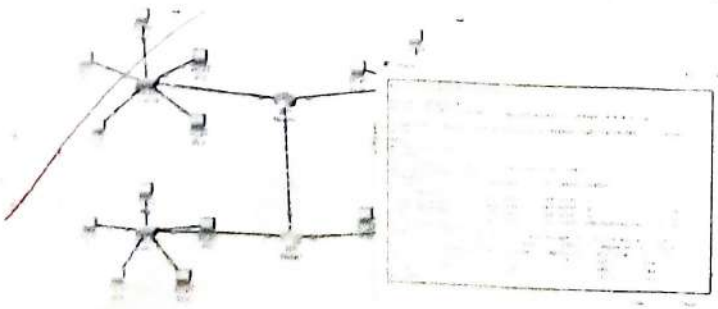
Once we have created our network topology we can add devices to it. Here, we will be adding routers, switches and PCs.

SUBNETTING

To subnet the network address of 192.168.1.0/24 to provide enough space for at least 5 addresses for end devices, the switch, router. we can use a /27 subnet mask This will give us 8 subnets with 30 host addresses each.









The IP addressing for the network shown in the topology can be as follows.

Router R1:

Gigabit Ethernet 0/0 : 192.168.1.1

Gigabit Ethernet 0/1 : 192.168.2.1

Switch S1:

Fast Ethernet 0/1 : 192.168.1.0/27

PC1 : 192.168.1.11

PC2 : 192.168.1.12

PC3 : 192.168.1.13

PC4 : 192.168.1.14

PC5 : 192.16.8.1.15

Fast Ethernet 0/2 : 192.168.2.0/27

PC1 : 192.168.2.11

PC2 : 192.168.2.12

PC3 : 192.168.2.13

PC4 : 192.168.2.14

PC5 : 192.168.2.15

Router R2:

Fast Ethernet 0/0 : 192.168.3.1

Fast Ethernet 0/1 : 192.168.4.1

Switch S2:

Fast Ethernet 0/2 : 192.168.4.0/27

PC1 : 192.168.4.11

PC2: 192.168.4.12
PC3: 192.168.4.13
PC4: 192.168.4.14
PC5: 192.168.4.15

Configure the devices

enable
configure terminal
interface FastEthernet 0/0
ip address [IP address] [subnet mask]
no shutdown
exit
interface FastEthernet 0/1
ip address [IP address] [subnet mask]
no shutdown
exit

Testing the network:

Now our topology is configured we can test the network. Open command prompt on each PC and try to ping the other PC. If the ping is successful, then the network is functioning properly.

Result

Thus subnetting in cisco packet tracer simulator is implemented successfully.

25/09/26

Exercise : 10

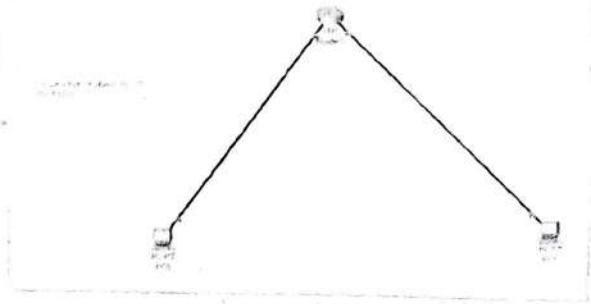
Internetworking with routers in CISCO PACKET TRACER simulator

a)

Aim :

Internetworking with routers in CISCO PACKET TRACER simulator.

In this network, a router and 2 PCs are used computers are connected with routers using a copper straight-through cable After forming the network, to check network connectivity a simple PDU is transferred from PC0 to PC1.



Procedure

Step 1 (configuring Router 1)

1. Select the router and open CLI.
2. Press ENTER to start configuring Router 1.
3. Type enable to activate the privileged mode

Router 1 command line interface

enable

config t

Enter configuration commands, one per line

interface FastEthernet 0/0

ip address 192.168.10.1

no shutdown

Step 2: Configuring PCs.

1. Assign IP addresses to every PC in network
2. Select PC, go to desktop and select IP configuration and assign an IP address, Default gateway subnet mask.
3. Assign the default gateway on PC0 as 192.168.10.1
4. Assign the default gateway of PC1 as 192.168.20.1

Step 3: Connecting PCs with router.

1. Connect FastEthernet0 port of PC0 with FastEthernet0/0 port of Router1 using a copper straight-through cable.
2. Connect FastEthernet0 port of PC1 with FastEthernet0/1 port of Router1 using a copper straight-through cable.

Router configuration table

Device name	IP address FastEthernet 0/0	Subnet mask	IP Address FastEthernet 0/1	Subnet mask
Router1	192.168.10.1	255.255.255.0	192.168.20.1	255.255.255.0

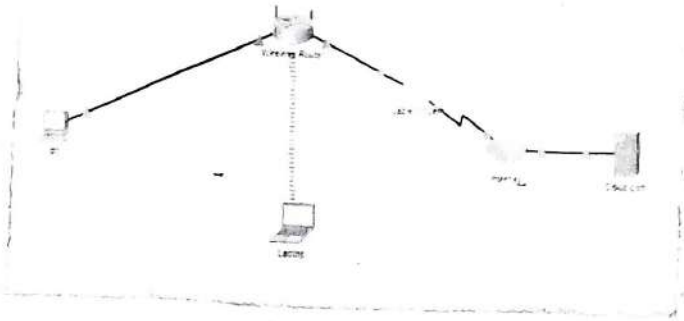
PC configuration table

Device name	IP address	Subnet mask	Gateway
PC0	192.168.10.2	255.255.255.0	192.168.10.1
PC1	192.168.20.2	255.255.255.0	192.168.20.1

b)

Aim:

Design and configure an internetwork using wireless router, DHCP server and, internet cloud.



Addressing Table

Device	Interface	IP address	Subnet mask	Default gateway
PC	Ethernet0	DHCP		192.168.0.1
wireless Router	LAN	192.168.0.1	255.255.255.0	
wireless Router	Internet	DHCP		
Cisco.com Server	Ethernet 0	208.67.220.220	255.255.255.0	
Laptop	Wireless 0	DHCP		

Part 1: Build a simple network in the Logical Topology Workspace

Part 2: Configure the Network Devices

Part 3: Test connectivity between network devices

Part 4: Save the File and close Packet Tracer

Part 1: Build a simple network

Step 1: Launch Packet Tracer

Step 2: Build the topology

Part 2: Configure the network devices

Step 1: Configure the wireless router

Step 2: Configure the laptop

Step 3: Configure the PC

Step 4: Configure the Internet cloud

Step 5: Configure the Cisco.com server

Part 3: Verify connectivity

Step 1: Refresh the IPV4 settings on the PC

Result:

Internetworking with routers in Cisco Packet Tracer simulator is done successfully.

28/09/24
Exercise : 11

a) Simulate Static Routing configuration using Cisco Packet Tracer

Aim:

To simulate and configure static routing using Cisco Packet Tracer

Theory:

Static routing is the process of manually adding routes to a router's routing table using the ip route command. A router automatically knows only its directly connected networks. Routes to remote networks must be configured manually. A lower Administrative Distance (AD) is preferred. Thus, a route with AD 10 acts as the main route and AD 20 acts as the backup route.

Procedure:

1. Create the required topology with 3 route and PCs in Cisco Packet Tracer.
2. Configure the IP addresses of all router interfaces and PCs.
3. Open the CLI of each router
4. Enter priviledged mode and global configuration mode
5. Configure static routes using the ip rout command
6. Use show ip route static to verify the routes.

7. Test connectivity using ping and tracert
8. Disconnect the main link and verify that the backup route is selected

Router 0 configuration

```
enable
configure terminal
ip route 30.0.0.0 255.0.0.0 20.0.0.2 10
ip route 30.0.0.0 255.0.0.0 40.0.0.2 20
ip route 30.0.0.100 255.255.255.255 40.0.0.2 10
ip route 30.0.0.100 255.255.255.255 20.0.0.2
ip route 50.0.0.0 255.0.0.0 40.0.0.2 10
ip route 50.0.0.0 255.0.0.0 20.0.0.2 20
exit
show ip route static
```

Router 1 configuration

```
enable
configure terminal
ip route 10.0.0.0 255.0.0.0 20.0.0.1 10
ip route 10.0.0.0 255.0.0.0 50.0.0.1 20
ip route 40.0.0.0 255.0.0.0 20.0.0.1 10
ip route 40.0.0.0 255.0.0.0 50.0.0.1 20
exit
show ip route static
```

Router 2 configuration

```
enable
configure terminal
ip route 10.0.0.0 255.0.0.0 40.0.0.1
ip route 30.0.0.0 255.0.0.0 50.0.0.2
exit
```

show ip route static

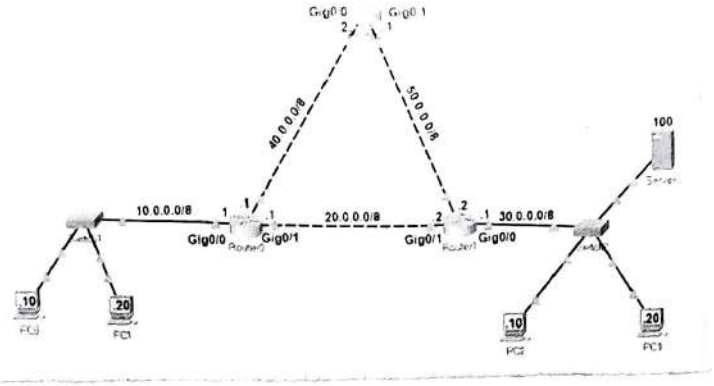
Verification

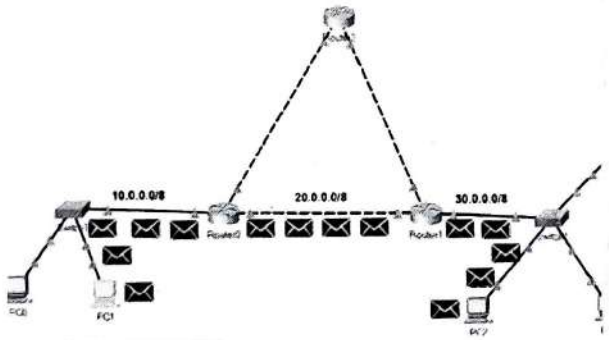
Use the following commands

show ip route static

ping <destination-IP>

tracert <destination-IP>





Physical Config CLI Attributes

IOS Command Line Interface

```
Router>enable
Router#show ip route static
      30.0.0.0/8 is variably subnetted, 2 subnets, 2 masks
S        30.0.0.0/8 [10/0] via 20.0.0.2
S        30.0.0.100/32 [10/0] via 40.0.0.2
S     50.0.0.0/8 [10/0] via 40.0.0.2

Router#
```

Physical Config Desktop Programming Attributes

Command Prompt

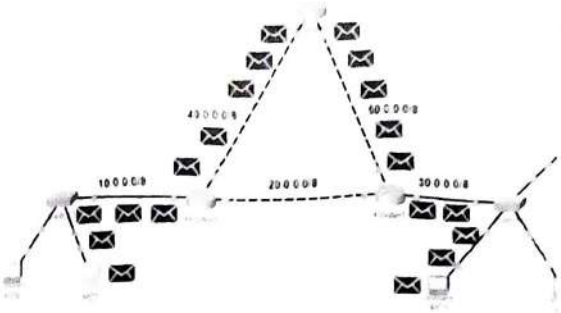
```
C:\>tracert 30.0.0.20

Tracing route to 30.0.0.20 over a maximum of 30
hops:

  1    0 ms      0 ms      0 ms      10.0.0.1
  2    0 ms      0 ms      0 ms      20.0.0.2
  3    *         0 ms      0 ms      30.0.0.20

Trace complete.

C:\>
```




```
Router>enable
Router#show ip route static
      30.0.0.0/8 is variably subnetted, 2 subnets, 2 masks
S        30.0.0.0/8 [10/0] via 20.0.0.2
S        30.0.0.100/32 [10/0] via 40.0.0.2
S     50.0.0.0/8 [10/0] via 40.0.0.2

Router#
```

```
Packet Tracer PC Command Line 1.0
C:\>tracert 30.0.0.100

Tracing route to 30.0.0.100 over a maximum of 30
hops:

  1   1 ms      0 ms      0 ms      10.0.0.1
  2   *         0 ms      0 ms      40.0.0.2
  3   *         *         0 ms      20.0.0.2
  4   *         0 ms      0 ms      30.0.0.100

Trace complete.

C:\>
```

```
Router>enable
Router#show ip route static
      30.0.0.0/8 is variably subnetted, 2 subnets, 2 masks
S        30.0.0.0/8 [10/0] via 20.0.0.2
S        30.0.0.100/32 [10/0] via 40.0.0.2
S     50.0.0.0/8 [10/0] via 40.0.0.2

Router#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#no ip route 30.0.0.100 255.255.255.255 40.0.0.2
Router(config)#exit
Router#
%SYS-5-CONFIG_I: Configured from console by console

Router#show ip route static
      30.0.0.0/8 is variably subnetted, 2 subnets, 2 masks
S        30.0.0.0/8 [10/0] via 20.0.0.2
S        30.0.0.100/32 [10/0] via 20.0.0.2
S     50.0.0.0/8 [10/0] via 40.0.0.2

Router#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#no ip route 30.0.0.100 255.255.255.255 20.0.0.2
Router(config)#exit
Router#
%SYS-5-CONFIG_I: Configured from console by console

Router#show ip route static
S     30.0.0.0/8 [10/0] via 20.0.0.2
S     50.0.0.0/8 [10/0] via 40.0.0.2

Router#
```

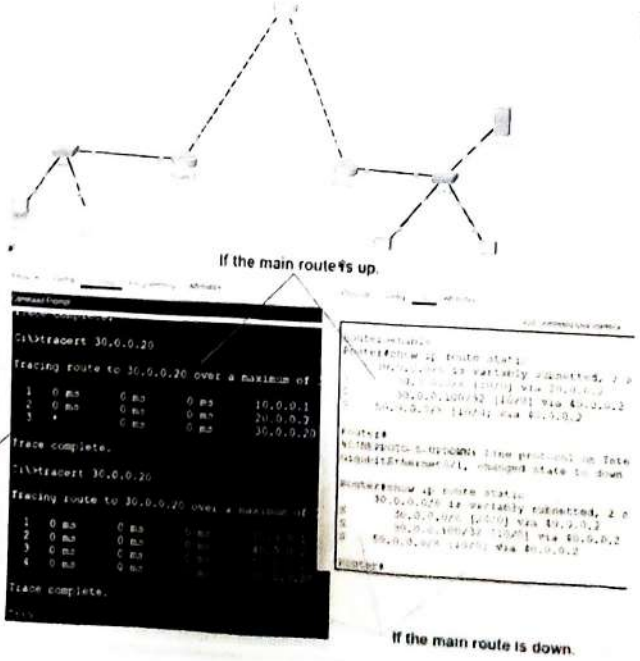
The main route that we want to delete.

Deleting the main route

As soon as we remove the main route, the router changes the backup route to the main route.

Deleting the new main route

All routes to host 30.0.0.100/8 have been removed.



If the main route is up.

If the main route is down.

b) Simulate RIP using Cisco Packet Tracer.

Aim:

To simulate and configure RIP dynamic routing using Cisco Packet Tracer

Theory:

RIP (Routing Information Protocol) is a dynamic routing protocol that automatically exchanges routing information between routers. RIP selects the route with the lowest hop count.

IP configuration

Device	Interface	IP Address
PC0	Fast Ethernet	10.0.0.2/8
Router 0	Fa0/1	10.0.0.1/8
Router 0	S0/0/0	192.168.1.249/30
Router 0	S0/0/1	192.168.1.254/30
Router 1	S0/0/0	192.168.1.246/30
Router 1	S0/0/1	192.168.1.245/30
Router 2	S0/0/0	192.168.1.250/30
Router 2	S0/0/1	192.168.1.253/30
Router 2	Fa0/1	20.0.0.1/8
PC1	Fast Ethernet	20.0.0.2/8

Procedure

1. Create the topology with 3 routers and 2 PCs
2. Assign IP addresses to PCs.
3. Configure IP addresses on all router interfaces
4. Use no shutdown to activate interfaces
5. Configure clock rate on the DCE side of

Serial connections
6 Enable RIP on all routers
7. Advertise the directly connected networks
8. Test connectivity using ping and tracert

RIP configuration

Router 0
enable
configure terminal
router rip
network 10.0.0.0
network 192.168.1.252
network 192.168.1.248

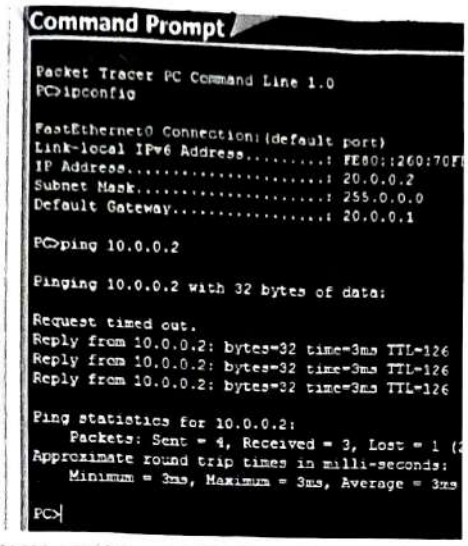
Router 1
enable
configure terminal
router rip
network 192.168.1.244
network 192.168.1.248

Router 2
enable
configure terminal
router rip
network 20.0.0.0
network 192.168.1.252
network 192.168.1.244

Verification

From PC0, test connectivity-

ping 20.0.0.2
tracert 20.0.0.2



Result

Thus routing at network layer is done successfully.

Exercise : 12

29/09/26

End to End communication at Transport Layer

Aim:

a) Implement echo client server using TCP/UDP sockets.

b) Implement a chat program using socket programming

TCP

```
import socket
import threading
def start_server():
    server = socket.socket(socket.AF_INET,
socket.SOCK_STREAM)
    server.bind(('127.0.0.1', 5000))
    server.listen(1)
    print("[SERVER] Listening on
127.0.0.1:5000")
    conn, addr = server.accept()
    print(f"[SERVER] connected by {addr}")
    while True:
        data = conn.recv(1024).decode()
        if not data or data.lower() ==
'quit':
            print("[SERVER] connection closed
            by client.")
            break
        print(f"[SERVER] Received: {data}")
```

```
        conn.send(f"Server received: {data}".
        encode())
    conn.close()
server.close()


def start_client():
    client = socket.socket(socket.AF_INET,
    socket.SOCK_STREAM)
    client.connect(('127.0.0.1', 5000))
    print("[CLIENT] connected to server")
    while True:
        msg = input("Enter message (or 'quit'
        to exit): ")
        client.send(msg.encode())
        if msg.lower() == 'quit':
            break
        response = client.recv(1024).decode()
        print(f"[SERVER RESPONSE] {response}")
client.close()
if __name__ == "__main__":
    server_thread = threading.Thread(targe
= start_server, daemon = True)
    server_thread.start
st start_client()
```

```
========== RESTART: C:/Users/tkala/Downloads/tcp.py ==========
[SERVER] Listening on 127.0.0.1:5000[CLIENT] Connected to server

[SERVER] Connected by ('127.0.0.1', 53458)Enter message (or 'quit' to exit):
hello
[SERVER] Received: hello
[SERVER RESPONSE] Server received: hello
Enter message (or 'quit' to exit):
========== RESTART: C:/Users/tkala/Downloads/tcp.py ==========
[SERVER] Listening on 127.0.0.1:5000[CLIENT] Connected to server

[SERVER] Connected by ('127.0.0.1', 60328)Enter message (or 'quit' to exit):
HI
[SERVER] Received: HI
[SERVER RESPONSE] Server received: HI
Enter message (or 'quit' to exit): quit
[SERVER] Connection closed by client.
```

UDP

```
import socket
import threading

def udp_server():
    s = socket.socket(socket.AF_INET,
    socket.SOCK_DGRAM)
    s.bind(('127.0.0.1', 12000))
    print("[SERVER] Running on
    127.0.0.1:12000")
    while True:
        data, addr = s.recvfrom(1024)
        msg = data.decode()
        if msg.lower()=='quit':
            print(f"[SERVER] client {addr}
            ended the connection.")
            break
        print(f"[SERVER] Received from {addr}:
```

```
        {msg}")
        s.sendto(data, addr)
    s.close()
    print("[SERVER] Stopped.")

def udp_client():
    c = socket.socket(socket.AF_INET,
    socket.SOCK_DGRAM)
    server = ('127.0.0.1', 12000)
    while True:
        msg = input("Enter message (or 'quit'
        to stop): ")
        c.sendto(msg.encode(), server)
        if msg.lower() == 'quit':
            print("[CLIENT] closing connection")
            break
        data, _ = c.recvfrom(1024)
        print("Echo from server: ", data.decode())
    c.close()
    print("[CLIENT] Stopped.")

if __name__ == "__main__":
    threading.Thread(target = udp_server,
    daemon = True).start()
    udp_client()
```

```
Enter message (or 'quit' to stop): [SERVER] Running on 127.0.0.1:12000hi

[SERVER] Received from ('127.0.0.1', 49670): hi
Echo from Server: hi
Enter message (or 'quit' to stop): hello
[SERVER] Received from ('127.0.0.1', 49670): hello
Echo from Server: hello
Enter message (or 'quit' to stop): how are you
[SERVER] Received from ('127.0.0.1', 49670): how are you
Echo from Server: how are you
Enter message (or 'quit' to stop): quit
[CLIENT] Closing connection...[SERVER] Client ('127.0.0.1', 49670) ended the connection.

[CLIENT] Stopped.[SERVER] Stopped.
```

Result:

End to end communication at transport layer is done and executed successfully.

05/10/26

Exercise : 13

Implement your own ping program

Aim:

To implement your own ping program

Code:

```
import os
import platform
import subprocess
def ping(host="google.com", count=4)
    param="-n" if platform.system().lower
    ()=="windows" else "-c"
    command=["ping", param, str(count),
    host]
    try:
        output=subprocess.check_output
        (command, universal_newlines=Tru
        print(output)
    except exception as e:
        print(f"Ping failed:{e}")
if __name__=="__main__":
    ping("8.8.8.8")
```

```
Pinging 8.8.8.8 with 32 bytes of data:
Reply from 8.8.8.8: bytes=32 time=6ms TTL=119
Reply from 8.8.8.8: bytes=32 time=9ms TTL=119
Reply from 8.8.8.8: bytes=32 time=5ms TTL=119
Reply from 8.8.8.8: bytes=32 time=9ms TTL=119

Ping statistics for 8.8.8.8:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 5ms, Maximum = 9ms, Average = 7ms
```

Result.

Thus own ping program was written and executed successfully.

06/10/26

Exercise: 14

Code using RAW sockets to implement packet sniffing

Aim:

To write a code using RAW sockets to implement packet sniffing.

Code:

```
from scapy.all import sniff
from scapy.layers.inet import IP, TCP, UDP, ICMP
def packet_callback(packet):
    if IP in packet:
        ip_layer = packet[IP]
        protocol = ip_layer.proto
        src_ip = ip_layer.src
        dst_ip = ip_layer.dst
        # Determine the protocol
        protocol_name = "
        if protocol == 1:
            protocol_name = "ICMP":
        elif protocol == 6 name =
            protocol_name = "TCP"
        elif protocol == 17:
            protocol_name = "UDP"
        else:
            protocol_name = "Unknown Protocol"
```

```
# Print packet details
print(f"Protocol : {protocol_name}")
print(f"Source IP : {src_ip}")
print(f"Destination IP : {dst_ip}")
print("-" * 50)
sniff(iface='Wi-Fi', prn=packet_callback,
filter="ip", store=0)
```



Result:

Thus a code using RAW sockets to implement packet sniffing is written and executed successfully.

Completed
13/10/26